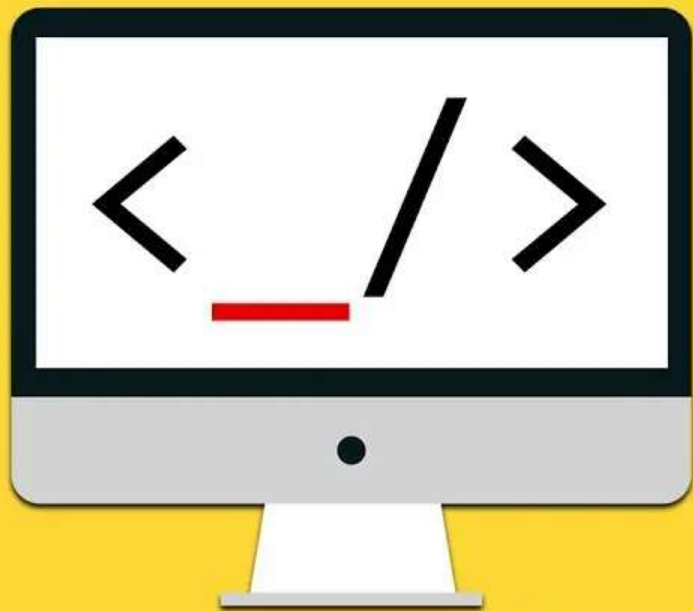




JavaScript

What is JavaScript

JavaScript is a high level programming language which is used to create interactive web pages.

It is the only language understood by browser.

JavaScript is a scripting language

It is the language you can use at browser and server sides.

It is the most commonly and popular language used right now.

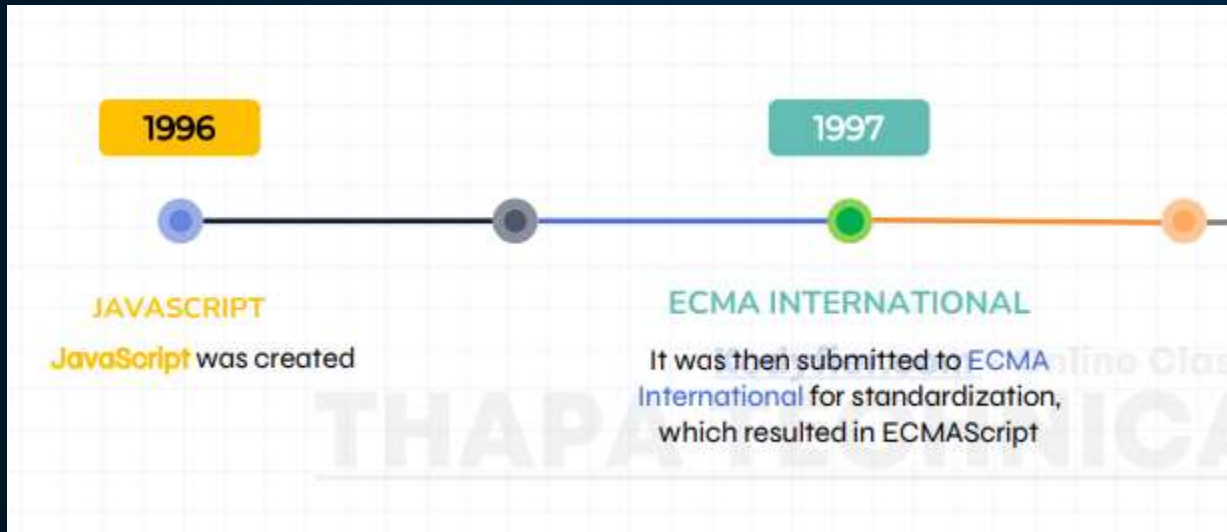
A lot of framework and libraries are based on Javascript. It can be used for both frontend and backend.

Frontend – React Js, Angular Js, Next Js

Backend - Node Js, Express Js

History of JavaScript

In 1995 - Created by Brendan Eich at Netscape in just 10 days..



History of JavaScript

Netscape Communication Corporations

Netscape Navigator(Browser)

Brandon Eich (1995)

JSE - SpiderMonkey

Mocha

LiveScript

Sun Microsystems

JavaScript

European Computer Manufacturers Association(ECMA)

ECMAScript

Versions Of EcmaScript



JavaScript Versions



Characteristics of JavaScript

- Client-side Scripting Language.
- Dynamically Typed Language.
- High Level Language.
- Weakly / loosely Typed Language.
- Interpreted Language / Synchronous / Single Threaded
- Object Based Language.
- Object Oriented Programing Language.
- Light-weight Language

Advantages of JavaScript

Client-Side Execution:

JavaScript runs directly in the browser, which allows for quick and responsive interactions without server-side communication, reducing server load and latency.

Versatility:

JavaScript can be used for front-end (React, Angular, Vue), back-end (Node.js), mobile development (React Native), and even desktop applications , making it a versatile language.

Cross-Browser Compatibility:

JavaScript runs on all modern web browsers, making it ideal for creating applications that work across different platforms without needing additional plugins.

Asynchronous Programming:

JavaScript's support for asynchronous programming (via Promises, async/await) helps create non-blocking code that improves performance, particularly in handling multiple tasks simultaneously.

Disadvantages of JavaScript

Security Vulnerabilities:

Code is Visible to everyone.

Browser Differences:

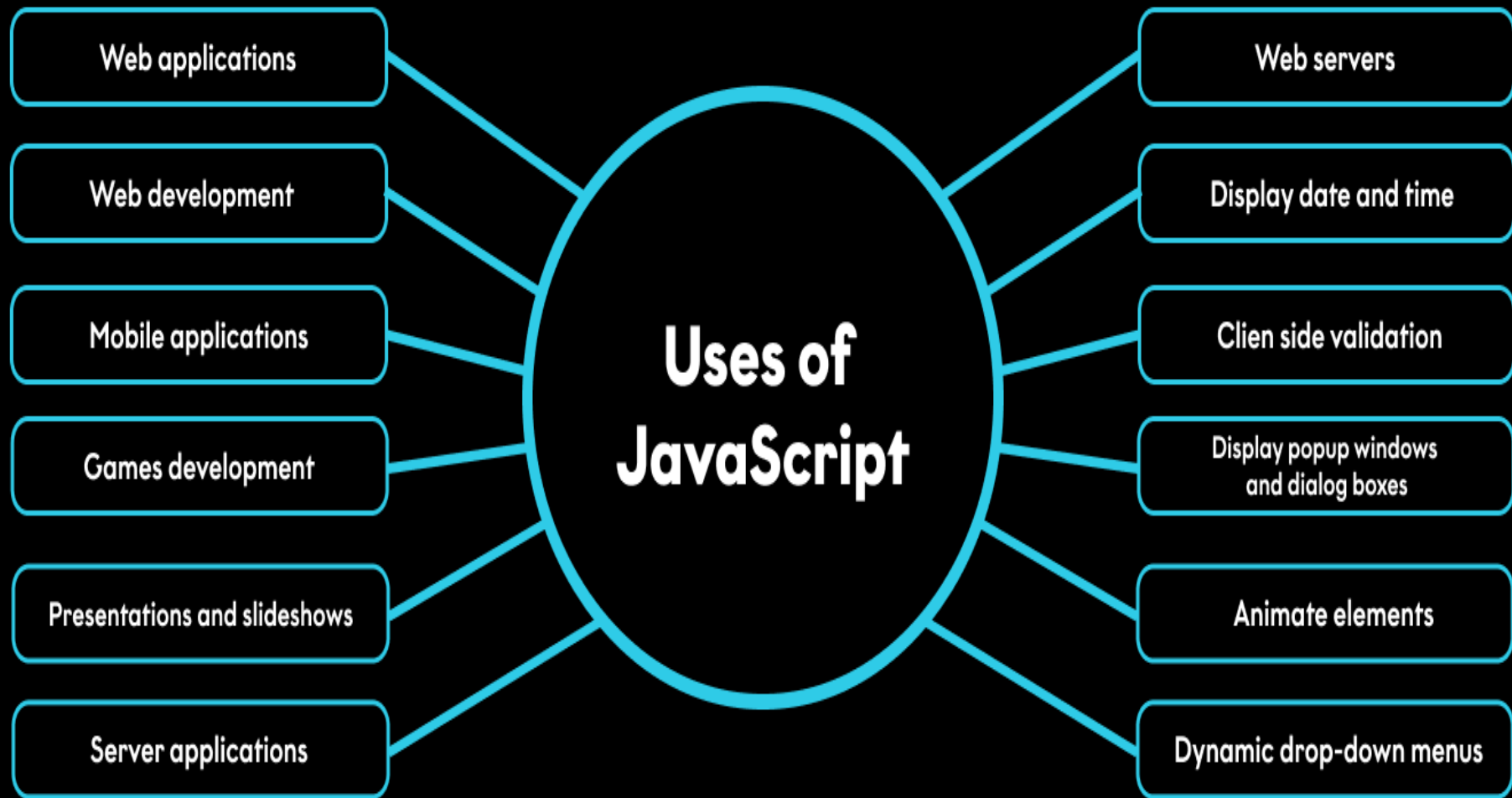
JavaScript behavior can vary across different browsers, requiring developers to test and possibly adjust their code for consistent behavior across platforms..

Loading Issue:

Interactive content may take longer to load

Lack of Static Typing:

JavaScript's lack of static typing means variable types are not fixed and can change at runtime, potentially leading to unexpected errors that are caught only when the code runs.



Difference between java and JavaScript

JAVA

Programing Language.

Multi-threaded.

Strictly Typed.

Runs on Java Virtual Machine.

More memory used.

Independent Language.

Both Compiled and Interpreted

JAVASCRIPT

Scripting Language.

Single-threaded.

Weekly Typed.

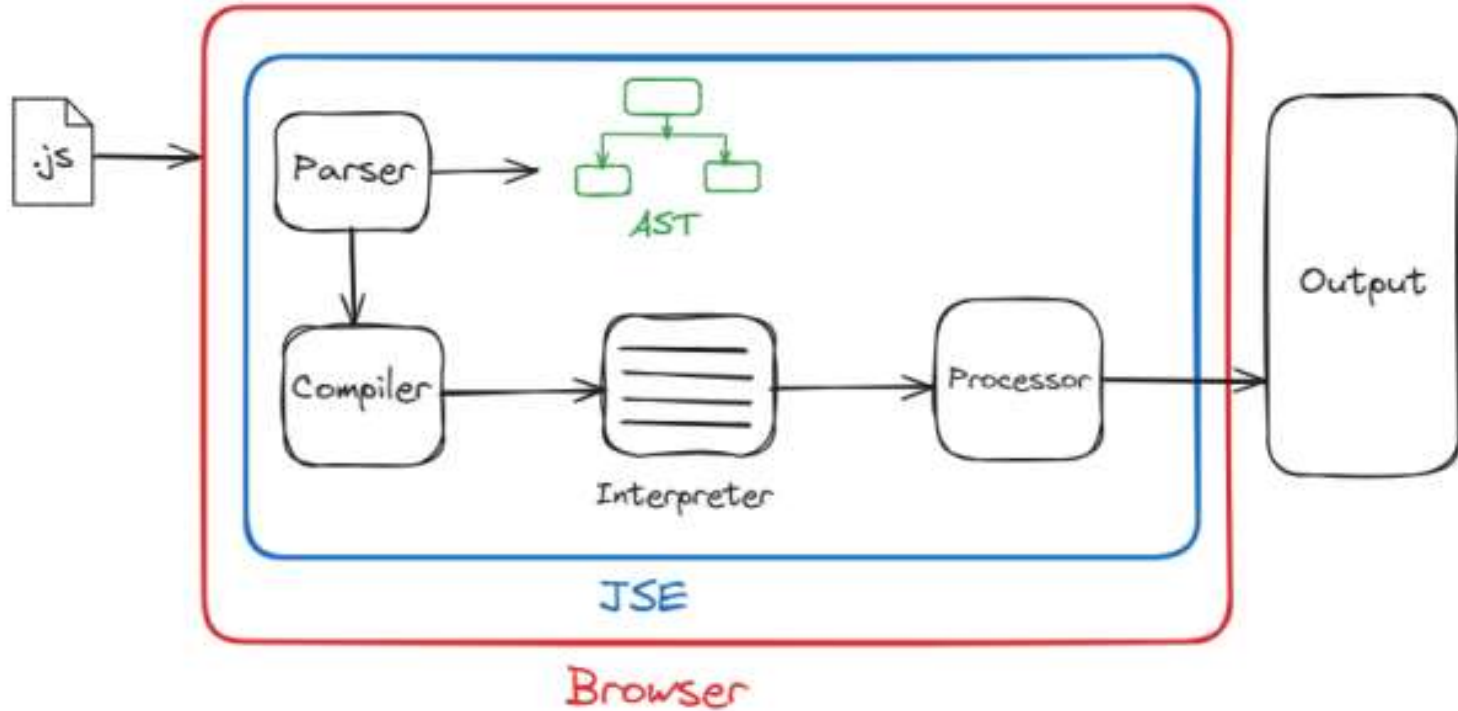
Runs on all the Browsers.

Less memory used.

Executed along with HTML.

Interpreted (not compiled)

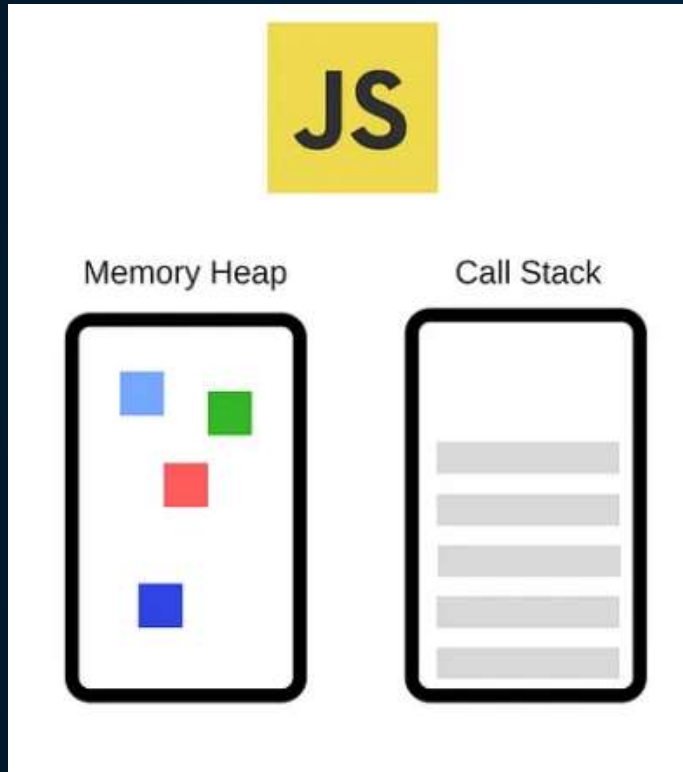
Working of JS Engine



JS Engines of Different Browser

Browser		Javascript Engine
Chrome		 V8 Engine (OpenSource)
Firefox		 Spider Monkey (OpenSource)
Safari		 JavaScriptCore (OpenSource)
Internet Explorer		 Chakra (Proprietary)

Working of JS Engine



The Engine consists of two main components:

Memory Heap — this is where the memory allocation happens

Call Stack — this is where your stack frames are as your code executes

Ways to write JavaScript

Internal JavaScript

```
<script> console.log('Hello, world!'); </script>
```



External JavaScript

```
<script src="script.js"></script>
```

How to Execute JavaScript

1. We can execute js instruction directly in the console provided by browser.
2. We can execute js instruction by embedding it in html page.

1.Internal: With the help of script tag we can embed js instruction.

Syntax:

```
<html >
  <head>
    <title>Document</title>
  </head>
  <body>
    <script>
      console.log("Hello World !!");
    </script>
  </body>
</html >
```

How to Execute JavaScript

2. External :

- We can create a separate file for js with extension .js
- Link the js file with html page using src attribute of script tag.

Syntax:

```
<html >
  <head>
    <title>Document</title>
  </head>
  <body>
    <script src="fileName.js "></script>
  </body>
</html >
```

Defer : Holds the execution of javascript file until complete html is loaded, works with external js only.

Tokens

Smallest unit of programming language.

Keywords: Predefined words which have some special meaning, understood by js, always in lowercase

Identifiers: Name provided to any variable, class, function or method.
It can't start with a number but can have number between it
It can't be a keyword
It can not have any special keyword other than \$, _
Does not contain space in between

Literals: Data provided by the user

Variables

A named block of memory which is used to store the value is known as variable.

We create variable using variable declaration followed by identifier.

Syntax:

```
variableDeclaration  identifierName : value;
```

There are three types of variable declaration:-

- 1.var
- 2.let
- 3.const

Variables

	Declaration	Initialization	Re-Declaration	Re-Initialization
Var	✓	✓	✓	✓
Let	✓	✓	✗	✓
Const	✓		✗	✗

Scope of Variables

Scopes

Scope is the visibility of a variable .

There are 4 types of scopes, They are:

1. Global Scope
2. Script Scope
3. Block
4. Function / Local Scope.

1. Global Scope:

If any variable is declared with var, This variable will store in Global Scope. We can access it from anywhere.

2. Script Scope:

If any variable is declared with let and const, this variable will store in script scope. We cannot access it at any time like before initialization.

Scope of Variables

3. Local Scope:

These are the variables which are defined for a specific scope like inside functions. These variables we cannot access outside the function.

Important Concept

Hoisting(***):

It is a process of accessing the variables before its initialization. This will be possible only if the variable is declared with var.

Temporal Dead Zone(***):

If any variable is declared with let or const, and we are trying to access before its initialization is known as Temporal Dead Zone. It returns an uncaught reference error.

Difference between var, let and const

Var	Let	Const
Var is global scope.	Let is block scope	Const is also block scope
We can declare multiple variable with same name(the most recently created variable Will be used.	We cannot declare 2 variables With the same name within a block.	We cannot declare 2 variables With the same name within a block.
The value of variable can be Modified.	The value of variable can be Modified.	The value of variable cannot be Modified.
<pre>vara=10; a=20; console.log(a); //20</pre>	<pre>let a=10; a=20; console.log(a);//20</pre>	<pre>const a=10; a=20; console.log(a);//we cannot modify the data with new value. //TypeError</pre>
We Can declare var without initialization. Eg:	We Can declare let without initialization. Eg:	We Cannot declare const without initialization. Eg:
<pre>var a;</pre>	<pre>let b;</pre>	<pre>const c; //Syntax error</pre>
The variable declared using var Belongs to global scope (window), We can access them using window Object.	The variable declared using let does Not belongs to global scope we cannot Use them with the help of window. {script /block}	The variable declared using const does Not belongs to global object we cannot Use them with the help of window. {script/block}
The variable declared using var is hoisted does not belong to temporal dead zone, can be used before initialization.	The variable declared using let is hoisted, the variable goes to temporal deadzone. Therefore it cannot be used before initialization.	The variable declared using const is hoisted, the variable goes to temporal deadzone. Therefore it cannot be used before initialization.
Declaring var inside function will not have global scope.		

Data Types

Primitive Data types

- 1.Number
- 2.String
- 3.Boolean
- 4.Null
- 5.Undefined
- 6.BigInt
- 7.Symbol

Non-Primitive Data types

- 1.Object

Data Types

Numbers - are values that can be processed and calculated. The numbers can be either integer or decimal values.

Strings - are a series of letters and numbers enclosed in quotation marks.

Boolean (true/false) - lets you evaluate whether a condition meets or does not meet specified criteria.

Null - is an empty value. null is not the same as 0. null is the absence of any value

Undefined-Represents a variable that has been declared but not assigned a value. It indicates the absence of an initial value.

Data Types

BigInt-A numeric type used for representing integers with arbitrary precision, allowing numbers larger than the Number type can handle ($2^{53}-1$)

Symbol-A unique and immutable primitive value often used as object property keys to create private or hidden properties in objects.

Object-A complex data type that allows you to store collections of data in key-value pairs.

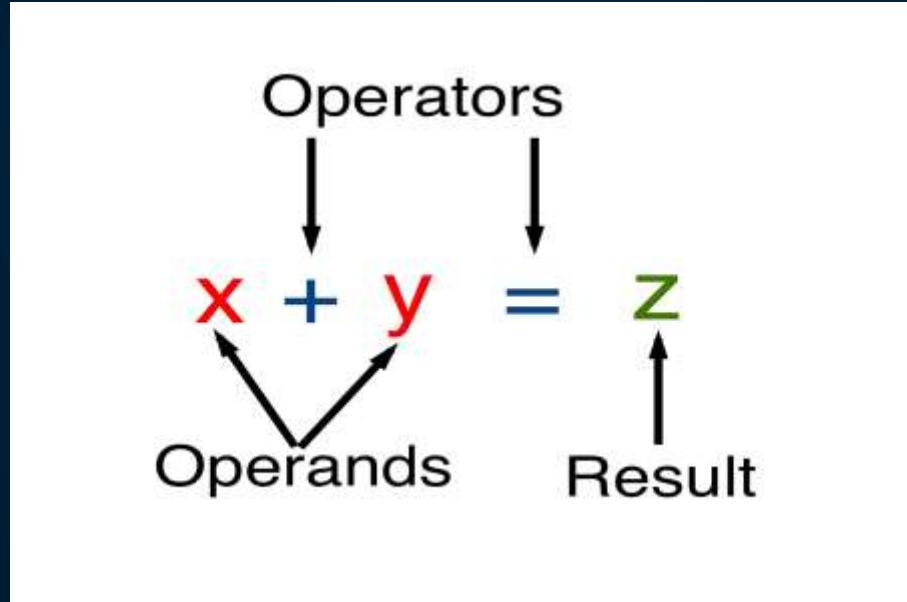
typeof : It is an unary operator used to check what type of data is present inside a variable, always return a String

Differences

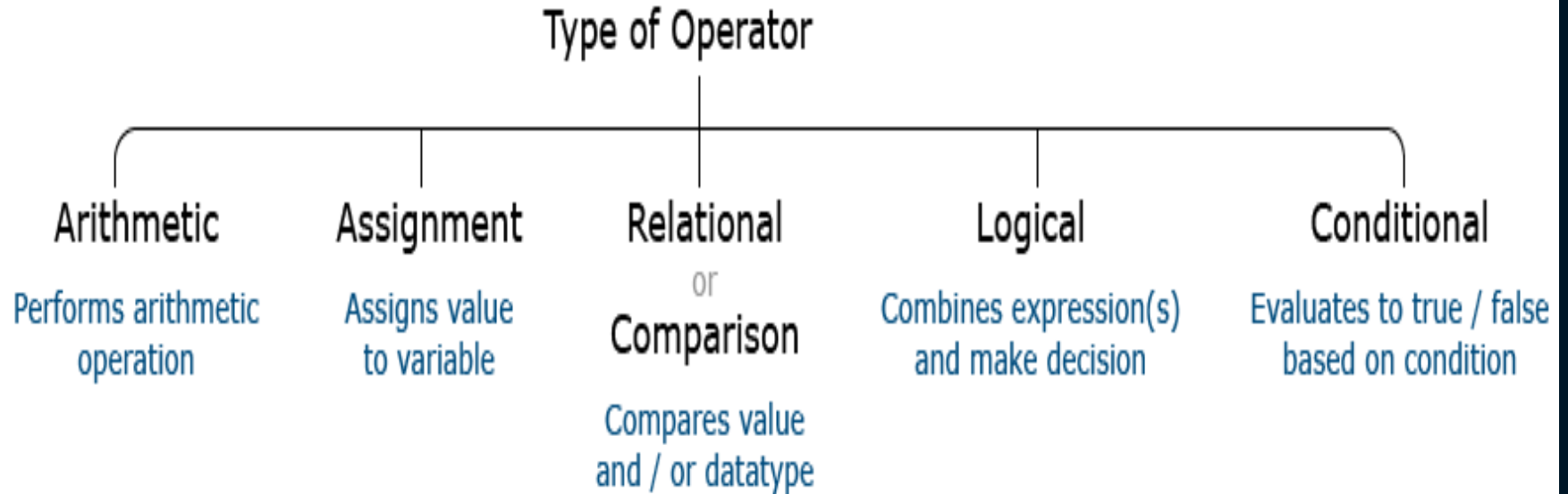
Primitive Data Types	Non-Primitive Data Types
Immutable (Cannot be changed)	Mutable (Can be changed)
Stored directly in the variable	Stored by reference
number, string, boolean, null, undefined, symbol	object, array
Checked by value	Checked by reference
Fixed size	Dynamic size
Simple operations (copy by value)	Complex operations (copy by reference)
Used for simple data values	Used for more complex structures and collections

Operators

Operators are the predefined symbols which are performing some specific tasks.



Types of Operators



Arithmetic Operators

Arithmetic operators are used to perform arithmetic between variables and/or values.

Given that $y=5$, the table below explains the arithmetic operators:

Operator	Description	Example	Result
+	Addition	$x=y+2$	$x=7$
-	Subtraction	$x=y-2$	$x=3$
*	Multiplication	$x=y*2$	$x=10$
/	Division	$x=y/2$	$x=2.5$
%	Modulus (division remainder)	$x=y\%2$	$x=1$
++	Increment	$x=++y$	$x=6$
--	Decrement	$x=--y$	$x=4$

Assignment Operators

Assignment operators are used to assign values to JavaScript variables

Given that $x=10$ and $y=5$, the table below explains the assignment operators:

Operator	Example	Same As	Result
=	$x=y$		$x=5$
+=	$x+=y$	$x=x+y$	$x=15$
-=	$x-=y$	$x=x-y$	$x=5$
=	$x=y$	$x=x*y$	$x=50$
/=	$x/=y$	$x=x/y$	$x=2$
%=	$x\%=y$	$x=x\%y$	$x=0$

Comparison Operators

Comparison operators are used in logical statements to determine equality or difference between variables or values.

Given that $x=5$, the table below explains the comparison operators:

Operator	Description	Example
<code>=</code>	is equal to	<code>x==8</code> is false
<code>===</code>	is exactly equal to (value and type)	<code>x===5</code> is true <code>x==="5"</code> is false
<code>!=</code>	is not equal	<code>x!=8</code> is true
<code>></code>	is greater than	<code>x>8</code> is false
<code><</code>	is less than	<code>x<8</code> is true
<code>>=</code>	is greater than or equal to	<code>x>=8</code> is false
<code><=</code>	is less than or equal to	<code>x<=8</code> is true

Logical Operators

Logical operators are used to determine the logic between variables or values.

Given that $x=6$ and $y=3$, the table below explains the logical operators::

Operator	Description	Example
&&	and	$(x < 10 \ \&\& \ y > 1)$ is true
	or	$(x==5 \ \ y==5)$ is false
!	not	$!(x==y)$ is true

Conditional/Ternary Operator

JavaScript also contains a conditional operator that assigns a value to a variable based on some condition.

Syntax :

```
variablename=(condition)?value1:value2
```

Decision Making Statement

1.If statements

2.If....else statements

3.Else....if ladder

4.Switch statements

Decision Making Statement

if statement : use this statement if you want to execute some code only if a specified condition is true

Syntax:

```
if (condition)
{
    code to be executed if condition is true
}
```

Decision Making Statement

If...else Statement- if you want to execute some code if a condition is true and another code if the condition is not true, use the if....else statement.

Syntax:

```
if (condition)
{
code to be executed if condition is true
}
else
{
code to be executed if condition is not true
}
```

Decision Making Statement

Else....if ladder: You should use the if....else if...else statement if you want to select one of many sets of lines to execute.

Syntax:

```
if (condition1)
{
code to be executed if condition1 is true
}
else if (condition2)
{
code to be executed if condition2 is true
}
else
{
code to be executed if condition1 and
condition2 are not true
}
```

Decision Making Statement

Switch case: You should use the switch statement if you want to select one of many blocks of code to be executed

Syntax:

```
switch(n)
{
  case 1:
    execute code block 1
    break;
  case 2:
    execute code block 2
    break;
  default:
    code to be executed if n is
    different from case 1 and 2
}
```

Looping Statement

It will iterate for n number of times

1. for loop

2. while loop

3. do-while loop

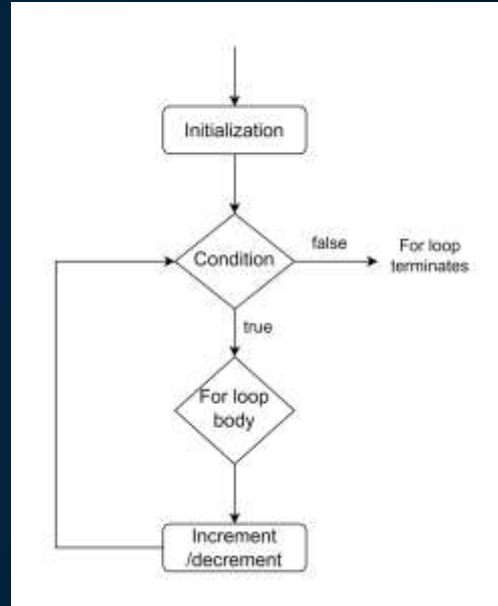
Looping Statement

for : used to execute a set of statements repetitively. It is commonly used when you know how many times you want to execute a block of code.

Entry-controlled loop

Syntax:

```
for (initialization; condition; updation){  
    // Code to be executed  
}
```



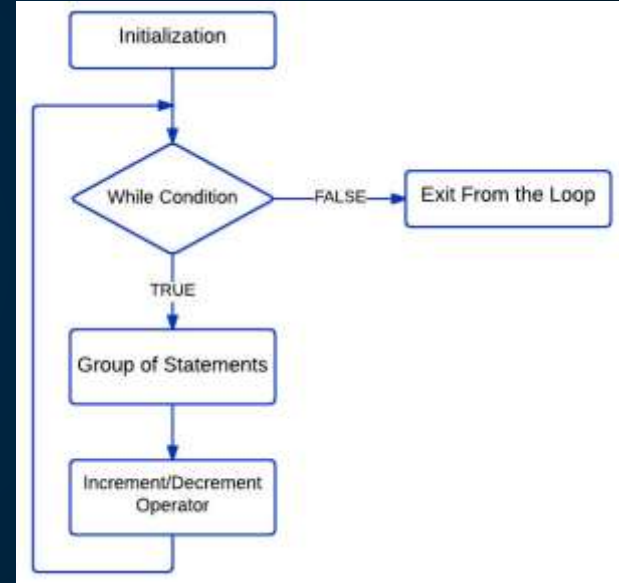
Looping Statement

while : loop loops through a block of code as long as a specified condition is true. It is commonly used when you don't know how many times you want to execute a block of code and it is based on condition

Entry-controlled loop

Syntax:

```
initialization
while (condition){
    // Code to be executed
    // updation
}
```



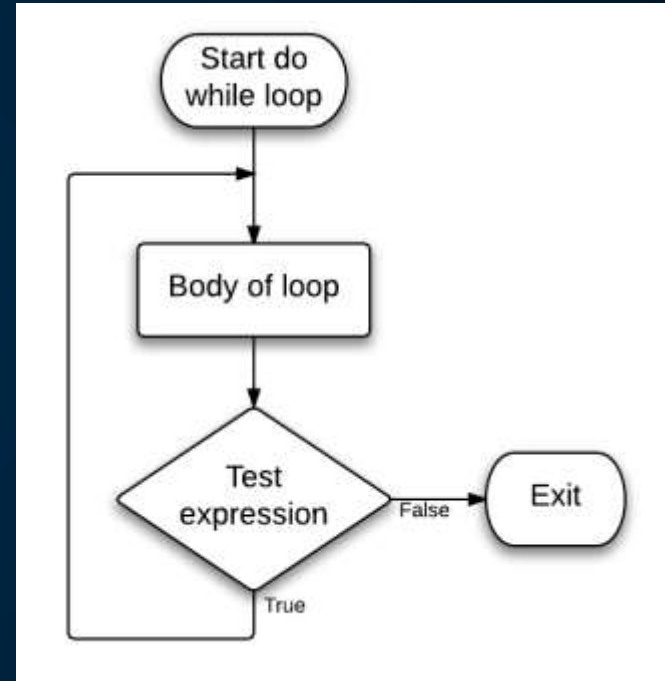
Looping Statement

do-while : This loop will execute the code block once, before checking if the condition is true, then it will repeat the loop as long as the condition is true. It is commonly used when you have to execute the loop atleast once.

Exit-controlled loop

Syntax:

```
initialization
do {
    // Code to be executed
    // updation
} while (condition)
```



Control flow of loop

Break - The break statement is used to jump out of a loop. It can be used to “jump out” of a switch() statement. It breaks the loop and continues executing the code after the loop.

Can only be used to jump out of a loop or switch

Continue - The continue statement “jumps over” one iteration in the loop. It breaks iteration in the loop and continues executing the next iteration in the loop.

Can only be used to skip one loop iteration

Type Conversion

Type conversion (or typecasting) means transfer of data from one data type to another.

Implicit Type Conversion - Implicit conversion happens when the compiler or runtime automatically converts data types.

Eg – $5 - '3' = 2$
 $5 + '3' = 53$

Explicit Type Conversion - Conversion from one data type to another knowing it might lead to data loss

Converting to Number - `Number()` - Returns a number, converted from its argument
`parseFloat()` - Parses a string and returns a floating point number
`parseInt()` - Parses a string and returns an integer

Converting to String - `String()` - Returns a string, converted from its argument
`toString()` - returns a string

Type Conversion

Converting to Boolean - Boolean() - Returns a boolean, converted from its argument

Falsy Values

- false
- 0
- "" or ' ' (Empty string)
- null
- undefined
- NaN

Truthy Values

- Everything else that is not falsy
- true
- '0' (0 in a string)
- ' ' (space in a string)
- 'false' (false in a string)
- [] (empty array)
- {} (empty object)
- function () {} (empty function)

Function

Function

In JavaScript, a function is a block of reusable code that performs a specific task.

It can take input parameters, perform operations, and return a value.

Functions are essential for structuring and organizing code, making it more readable, maintainable, and efficient.

Function

Function Declaration

A function definition (also called a function declaration, or function statement) consists of the function keyword, followed by:

1. The name of the function.
2. A list of parameters to the function, enclosed in parentheses and separated by commas.
3. The JavaScript statements that define the function, enclosed in curly braces, { /* ... */ }.

Ex : `function square(number) {
 return number * number;
 }`

Function

Parameters and Arguments

Parameters - Function parameters are the variable listed in the function definition.

Arguments - Function arguments are the real values passed to (and received by) the function.

```
function functionName(parameter1, parameter2, parameter3) {  
    // code to be executed  
}  
functionName(argument1, argument2, argument3)
```

Parameter rules –

JavaScript function definitions do not specify data types for parameters.

JavaScript functions do not perform type checking on the passed arguments.

JavaScript functions do not check the number of arguments received.

Function

Default Parameters –

If a function is called with missing arguments (less than declared), the missing values are set to undefined, we can pass default values also.

```
function myFunction(x, y = 10) {  
  return x + y;  
}  
myFunction(5);
```

Function

Return

The JavaScript return statement is used to stop the function's execution and optionally return a value to the caller. When a return statement is encountered, the function stops running and outputs the specified value, allowing the function to pass data back. It can be used to return a specific value, variable, or expression to the caller. If no value is provided, the function returns undefined by default.

Syntax – return value;
value: The value returned to the function caller. It is an optional parameter. If the value is not specified, the function returns undefined

```
function Product(a, b) {  
    // Return the product of a and b  
    return a * b;  
};  
console.log(Product(6, 10));
```

Function

Invocation

The code inside a function is not executed when the function is defined.

The code inside a function is executed when the function is invoked.

It is common to use the term "call a function" instead of "invoke a function".

It is also common to say "call upon a function", "start a function", or "execute a function".

```
function myFunction(a, b) {  
  return a * b;  
}  
myFunction(10, 2);    // return 20
```

Function

Types Of Functions

1. Named Function
2. Anonymous Function
3. First Class Function
4. Function with expression
5. Immediate invoke Function Expression (IIFE)
6. Arrow Function
7. Higher order Function
8. Callback Function
9. Nested Function
10. Generator Function

Function

1. Named Function

A function declared with an identifier name is called Named Function.

Syntax:

```
function fName(params) {  
    //block of code  
}  
fName(args) //function Call
```

Ex:

```
function sum(a,b){  
    return a+b;  
}  
console.log(sum(10, 20)); // 30
```

Function

2. Anonymous Function:

A function declared without an identifier is called Anonymous Function.

To execute anonymous functions, we have to store them in a variable.

Syntax :

```
var variableName = function (params){  
    // Code  
}  
variableName(args)  
Ex: var add = function (a, b) {  
    return a + b;  
};  
console.log(add(10, 20));
```

Function

3. First Class Function:

It is a function which is assigned as a value to a variable.

It can be named Function / anonymous function / arrow function.

It can be accessed only with the variable name , you cannot access it with the function name

Ex :

```
let demo = function add(a,b){  
    console.log(a+b);  
}  
demo(20,10) // 30;  
add(20,10) // Error
```

4. Functional Expression: An expression of storing any function into a variable.

Function

5. Immediate Invoke Functional Expression(IIFE):

This function consists of a couple of parentheses followed by a semicolon.

The first parenthesis indicates the function.

The second parenthesis indicates the function invocation.

semi colon indicates end of function.

Syntax:

```
(function_Area)(function_Invokation);
```

Ex 1:

```
(function (a, b) {  
    console.log(a + b);  
})(10, 20);
```

Ex 2:

```
(a => console.log(a ** 2))(10);
```

Function

6. Arrow Function:

It is an advanced function, It will reduce the code.

To execute this function we have to store it in a variable.

Syntax:

```
() => { //code } /
```

Ex:

```
var s = (a) => {  
    return a ** 2;  
};  
console.log(s(10));
```

Function

Implicit return :

If we have only one statement in a block then the block is not required and we can return it in the same line without using return keyword

Ex :

```
var s = (a) => a ** 2;  
console.log(s(10)); // 100
```

If we have only one parameter then parenthesis is not required.

Ex :

```
var s = a => a ** 2;  
console.log(s(10)); // 100
```

Function

7. Higher Order Function:

It is a function which accepts another function as an argument. It will return a function.

8. Callback Function: It is a function which sends as a parameter to another function.

```
Ex: function Cal(a, b, operation) {  
    return operation(a, b);  
}  
  
function sum(a, b) {  
    return a + b;  
}  
  
function sub(a, b) {  
    return a - b;  
}  
  
function mul(a, b) {  
    return a * b;  
}  
  
console.log(Cal(10, 20, sum)); // 30  
console.log(Cal(10, 20, sub)); // -10  
console.log(Cal(10, 20, mul)); // 200
```

In this example Cal is Higher order function and sum , sub , mul are callback functions

Function

Generator Functions:

Functions that can pause and resume execution, yielding values in the process.

```
function* generateNumbers() {  
    yield 1;  
    yield 2;  
    yield 3;  
}
```

```
let serial = generateNumbers();  
console.log(serial.next().value)  
console.log(serial.next().value)
```

Function

9. Nested Function:

A function inside another function is called a nested function.

Syntax:

```
function fun1(){  
    // Code  
    function fun2(){  
        // Code  
    }  
    // Code }  
}
```

Function

Ex:

```
function Bank(){
    var accountNo = 123;
    var holderName = "Akshay Thakur";
    var balance = 50000;
    function Deposit(){
        var amount = Number(prompt("Enter amount:"));
        return amount + balance;
    }
    return Deposit;
}
console.log(Bank());
```

Function

Closure:

Whenever the nested function requires the variable of the parent function, JSE will create closures.

Closures will be holded by Child function. Closures will hold only required variables.

Curing:

It is a process of invoking the nested functions along with the parent function. In the above example Bank()() is called curing.

Array



Ram



Hari



Sita



Bishal



Gita

Imagine you want to store collection of people names.

Array

const person1 = Ram

const person2 = Hari

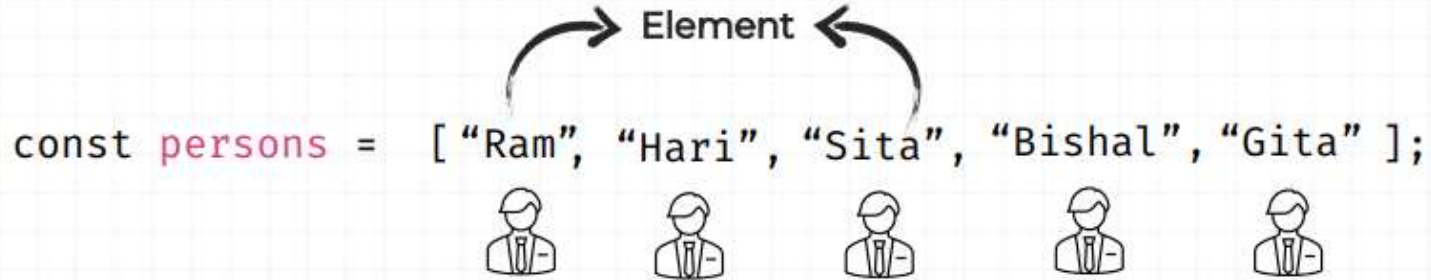
const person3 = Sita

const person4 = Bishal

const person5 = Gita



Array



The diagram shows a code snippet `const persons = [ "Ram", "Hari", "Sita", "Bishal", "Gita" ];` on a grid background. Above the array elements, the word "Element" is centered with two curved arrows pointing to the first and last elements. Below each of the five string elements, there is a simple line-art icon of a person wearing a suit and tie.

```
const persons = [ "Ram", "Hari", "Sita", "Bishal", "Gita" ];
```

Array:

Array is an object.

It enables storing a collection of multiple items under a single variable name.

Ways to create an Array

1.Literal way:

Arrays can be created using the literal notation.

Syntax:

VariableDeclaration arrayReferenceVariable=[element1,element2,,,,,];

Example-

```
const fruits = ["Apple", "Banana"];
```

Ways to create an Array

2. Constructor way:

Arrays can be created using Array Constructor.

Syntax:

```
VariableDeclaration arrayReferenceVariable=new Array(element1,  
element2, /* ..., */ element)
```

Example-

```
const fruits = new Array("apple","banana");
```

Methods of Array

length:

The length property returns the length (size) of an array:

Syntax: arr_ref.length

Ex:

```
const fruits = ["Ram", "Hari", "Sita", "Bishal"];  
var a=fruits.length  
console.log(a);  
4 //Output
```

Methods of Array

`at()`:

The `at()` method returns an indexed element from an array

Syntax: `arr_ref.at(index)`

Return Type: The element in the array matching the given index.

Ex:

```
const names = ["Ram", "Hari", "Sita", "Bishal"];  
var a=names.at(2)  
console.log(a);  
Sita //Output
```

Methods of Array

Push():

Push Method: The Push Method that adds one or more elements to the end of an array.

Syntax: arr_ref. push(Element)

Return Type: It returns the new length of array.

Ex:

```
persons.push('Gita')
```

Index	Number	=	0	1	2	3
const persons	=		["Ram",	"Hari",	"Sita",	"Bishal"];
						

Gita added at the end of the array

```
persons.push('Gita')
```

Index	Number		0	1	2	3	4
const persons	=		["Ram",	"Hari",	"Sita",	"Bishal",	"Gita"];
							

Methods of Array

`pop()`:

Pop Method: Method that removes the last element from an array.

Syntax: `arr_ref. pop()`

Return Type: The removed element from the array.

Ex:

```
Index Number      0      1      2      3      4
const persons = [ "Ram", "Hari", "Sita", "Bishal", "Gita" ];
```

Gita will be removed

`persons.pop('Gita')`

Methods of Array

shift():

The shift() method removes the first array element.

Syntax: arr_ref. shift()

Return Type: The removed element from the array.

Ex:

```
const fruits = ["Ram", "Hari", "Sita", "Bishal"];
fruits.shift();
Console.log(fruits)
["Hari", "Sita", "Bishal"]; // output
```

Methods of Array

unshift():

The unshift() method adds a new element to an array (at the beginning)

Syntax: arr_ref.unshift(element)

Return Type: It returns the new length of array.

Ex:

```
const fruits = ["Ram", "Hari", "Sita", "Bishal"];
fruits.unshift("Gita");
Console.log(fruits)
["Gita", "Ram", "Hari", "Sita", "Bishal"]; // output
```

Methods of Array

`concat()`:

The `concat()` method creates a new array by merging (concatenating) existing arrays

Syntax: `arr_ref.concat(newarrayreferencevariable)`

Return Type: It returns a new array.

Ex:

```
const fruits = ["Ram", "Hari", "Sita", "Bishal"];  
const arr=["Raj"]  
fruits.concat(arr)  
["Gita", "Ram", "Hari", "Sita", "Bishal", "Raj"]; // output
```

Methods of Array

`toString()`:

`toString()` converts an array to a string of (comma separated) array values.

Syntax: `arr_ref.toString()`

Return Type: A string representing the elements of the array.

Ex:

```
const fruits = ["Ram", "Hari", "Sita", "Bishal"];  
var a=fruits.toString()  
console.log(a);  
Ram,Hari,Sita,Bishal //Output
```

Methods of Array

`join()`:

The `join()` method also joins all array elements into a string separated by commas.

Syntax: `arr_ref.join()`

Return Type: A string with all array elements joined.

Ex:

```
const fruits = ["Ram", "Hari", "Sita", "Bishal"];
```

```
fruits.join()
```

```
Ram,Hari,Sita,Bishal // output
```

Methods of Array

`flat()`:

The `flat()` method creates a new array with sub-array elements concatenated to a specified depth.

Syntax: `arr_ref.flat()`

Return Type: A new array with the sub-array elements concatenated into it..

Ex:

```
const fruits = [ ["Ram"], [ "Hari"], ["Sita", "Bishal"] ];  
fruits.flat()  
['Ram', 'Hari', 'Sita', 'Bishal'] // output
```

Methods of Array

splice():

The splice() method can be used to add new items to an array:

Syntax: arr_ref.splice(start, deleteCount, item1, item2, /* ..., */ itemN)

Start: At which index to start changing the array.

deleteCount: An integer indicating the number of elements in the array to remove from start

Items: The elements to add to the array, beginning from start.

Return Type: An array containing the deleted elements.

Methods of Array

Ex:

```
const fruits = ["Ram", "Hari", "Sita", "Bishal"];  
var a=fruits.splice(1,1,"Raj")
```

```
console.log(a); ['Hari'] //Output  
console.log(fruits);  
['Ram', 'Raj', 'Sita', 'Bishal'] // Output
```

Methods of Array

slice():

The slice() method slices out a piece of an array into a new array:

Syntax: arr_ref.slice(start,end)

Start: It specifies from which index it will start to create new array.

end: It specifies the array to be created before this index.

Return Type: A new array containing the extracted elements.

Ex:

```
const fruits = ["Ram", "Hari", "Sita", "Bishal"];  
var a=fruits.slice(1,3)  
console.log(a);  
['Hari', 'Sita'] //Output
```

Methods of Array

`reverse()`:

The `reverse()` method reverses the elements in an array:

Syntax: `arr_ref.reverse()`

Return Type: The reference to the original array, now reversed.

Ex:

```
const fruits = ["Ram", "Hari", "Sita", "Bishal"];  
var a=fruits.reverse()  
console.log(a);  
['Bishal', 'Sita', 'Hari', 'Ram'] //Output
```

Methods of Array

`sort()`:

The `sort()` method sorts an array alphabetically

Syntax: `arr_ref.sort()`

Return Type: The reference to the original array, now sorted.

Ex:

```
const fruits = ["Ram", "Hari", "Sita", "Bishal"];  
var a=fruits.sort()  
console.log(a);  
['Bishal', 'Hari', 'Ram', 'Sita'] //Output
```

Methods of Array

map():

The map() method creates a new array by performing a function on each array element.

The map() method does not execute the function for array elements without values.

The map() method does not change the original array.

** This methods accepts arguments as a call back function.

Syntax: arr_ref.map(callbackFunction(value, index, array){
 //perform task on each elements
})

Methods of Array

value (or currentValue): The current element being processed in the array.

index: The index of the current element being processed.

array: The original array on which the map method was called

Return Type: A new array with each element being the result of the callback function.

Ex:

```
const arr = [1,2,3,5,8];
```

```
var a=arr.map((value)=>{  
    return value**2  
})  
console.log(a);
```

```
[1, 4, 9, 25, 64] //Output
```

Methods of Array

`filter()`:

The `filter()` method creates a new array with array elements that pass a condition.

****** This methods accepts arguments as a call back function.

Syntax: `arr_ref.filter(callbackFunction(value){
 //perform filter on each elements
})`

Note: The callback function should return a truthy value to keep the element in the resulting array, and a falsy value otherwise.

Methods of Array

Return Type: A new array(Shallow Copy)

Ex:

```
const arr = [1,2,3,5,8];
```

```
var a=arr.filter((value)=>{  
  return value>3  
})
```

```
console.log(a);
```

```
[5, 8] // Output
```

Methods of Array

reduce():

The reduce() method runs a function on each array element to produce (reduce it to) a single value.

The reduce() method works from left-to-right in the array.

** This methods accepts arguments as a call back function.

Syntax: arr_ref.reduce(callbackFunction(total,value){
 //perform reduce operation
},initialValue)

Total: the initial value / previously returned value

Initital value: The value which we can store initially in total.(Optional)

Methods of Array

Return Type: The final result after processing all elements.

Ex:-

```
const arr = [1,2,3,5,8];
```

```
var a=arr.reduce((total,value)=>{  
    return total+value;
```

```
},5)
```

```
console.log(a);
```

```
24 // output
```

Methods of Array

forEach():

The forEach() method calls a function (a callback function) once for each array element.

****** This methods accepts arguments as a call back function.

Return Type: None

Syntax: arr_ref.forEach(callbackFunction(value){
 //performs operation on each element
})

Ex:

```
const arr = [1,2,3,5,8];  
var a=arr.forEach((value)=>{  
    console.log(value*2);  
})
```

O/P

2

4

6

10

16

For of Loop

For of:

The JavaScript for of statement loops through the values of an iterable object.

Syntax:

```
for (variable of iterable) {  
  // code block to be executed  
}
```

variable - For every iteration the value of the next property is assigned to the variable. Variable can be declared with const, let, or var.

iterable - An object that has iterable properties

For of Loop

Ex:

```
const arr = [1,2,3,5,8];
```

```
for(let x of arr){  
  console.log(x);  
}
```

O/P

1

2

3

5

8

Strings

String:

Strings are the sequence of characters.

Strings are written with quotes

Strings created with single or double quotes works the same.

There is no difference between the two.

Ways to create Strings

1.String literals:

Strings can be created as primitives, from string literals.

Syntax: VariableDeclaration VariableName="Sequence of characters"

or

VariableDeclaration VariableName='Sequence of characters'

Ex:

```
let text="ramsita"
```

```
let text='ramsita'
```

Ways to create Strings

2.String Constructor:

Strings can be created as an object by using String() constructor.

Syntax: VariableDeclaration VariableName=new String("sequence of character")

Ex:

```
let text=new String("ramsita")
```

Note: strings are automatically converted to string objects when using string methods on them. This process is called auto-boxing.

Methods of Strings

Length:

The length property returns the length of a string:

Syntax: `str_ref.length`

Ex:

```
let text="Babita Jethalal"  
console.log(text.length);
```

15 // Output

Methods of Strings

at():

Returns the character at the specified index.

Syntax: str_ref.at(index)

Return Type: A String consisting of the single character.

Ex:

```
let text="Babita Jethalal"  
console.log(text.at(1));
```

a // Output

Note: It Accepts negative Value as parameters

Methods of Strings

`charAt()`:

Returns the character at the specified index.

Syntax: `str_ref.charAt(index)`

Return Type: A String consisting of the single character.

Ex:

```
let text="Babita Jethalal"  
console.log(text.charAt(1));
```

a // Output

Methods of Strings

charCodeAt():

The charCodeAt() method returns the code of the character at a specified index in a string:

Syntax: str_ref.charCodeAt(index)

Return Type: Ascii Value.

Ex:

```
let text="Babita Jethalal"  
console.log(text.charCodeAt(1));
```

97 // Output

Methods of Strings

`concat()`:

`concat()` joins two or more strings:

Syntax: `str_ref1.concat(str_ref2)`

Return Type: A new string containing the combined text of the strings provided.

Ex:

```
let text1="Babita"  
let text2="Jethalal"  
console.log(text1.concat(text2));
```

BabitaJethalal // Output

Methods of Strings

`trim()`:

The `trim()` method removes whitespace from both sides of a string:

Syntax: `str_ref.trim()`

Return Type: A new string without white space from both sides.

Ex:

```
let text="      Babita Jethalal      "  
console.log(text.trim());
```

Babita Jethalal // Output

Methods of Strings

`trimStart()`:

The `trimStart()` method removes whitespace from starting of a string:

Syntax: `str_ref.trimStart()`

Return Type: A new string without white space from starting sides.

Ex:

```
let text="      Babita Jethalal      "  
console.log(text.trimStart());
```

Babita Jethalal // Output

Methods of Strings

`trimEnd()`:

The `trimEnd()` method removes whitespace from ending of a string:

Syntax: `str_ref.trimEnd()`

Return Type: A new string without white space from ending sides.

Ex:

```
let text="      Babita Jethalal      "  
console.log(text.trimEnd());
```

Babita Jethalal // Output

Methods of Strings

toUpperCase():

A string is converted to upper case with toUpperCase():

Syntax: str_ref.toUpperCase()

Return Type: A new string representing the calling string converted to upper case.

Ex:

```
let text="Babita Jethalal"  
console.log(text.toUpperCase());
```

BABITA JETHALAL // Output

Methods of Strings

`toLowerCase()`:

A string is converted to lower case with `toLowerCase()`:

Syntax: `str_ref. toLowerCase()`

Return Type: A new string representing the calling string converted to lower case.

Ex:

```
let text="Babita Jethalal"  
console.log(text. toLowerCase());
```

`babita jethalal` // Output

Methods of Strings

slice():

slice() extracts a part of a string and returns the extracted part in a new string

Syntax: str_ref. slice(start,end)

Return Type: A new string containing the extracted section of the string.

Ex:

```
let text="Babita Jethalal"  
console.log(text.slice(3,7));
```

ita // Output

Note: It will also extract the end index

Methods of Strings

substring():

substring() extracts a part of a string and returns the extracted part in a new string

Syntax: str_ref.substring(start,end)

Return Type: A new string containing the extracted section of the string.

Ex:

```
let text="Babita Jethalal"  
console.log(text.slice(3,7));
```

ita // Output

Note: The difference is that start and end values less than 0 are treated as 0

Methods of Strings

replace():

The replace() method replaces a specified value with another value in a string:

Syntax: str_ref.replace("oldcharacter","newcharacter")

Return Type: A new string by replacing the first occurrence.

Ex:

```
let text="Babita Jethalal"  
console.log(text.replace("a","z"));
```

Bzbita Jethalal // Output

Methods of Strings

replaceAll():

The replaceAll() method replaces a specified value with another value in a string:

Syntax: str_ref.replaceAll("oldcharacter","newcharacter")

Return Type: A new string by replacing the all occurrence.

Ex:

```
let text="Babita Jethalal"  
console.log(text.replaceAll("a","z"));
```

Bzbitz Jethzlzl // Output

Methods of Strings

`split()`:

A string can be converted to an array with the `split()` method:

Syntax: `str_ref.split(separator)`

Separator: The pattern describing where each split should occur

Return Type: An Array of strings

Ex:

```
let text="Babita Jethalal"  
console.log(text.split(" "));
```

`['Babita', 'Jethalal']` // Output

Object

Objects are real-time entities that have properties and methods.

Objects are defined by using {}.

We will store the data inside the object in the form of key-value pair.

```
Ex :- let details = {  
      name: "dinga",  
      id : 1  
    }
```

Ways to create object

1.Object Literals:

Object literals are a comma-separated list of key-value pairs wrapped in curly braces. As objects are variables, we can instantiate them the same way as a variable.

```
var identifier = {  
    key1 : value1,  
    key2 : value2,  
    key3 : value3,  
    .  
    .  
    .  
    key_n : value_n  
}
```

Ways to create object

2. Object Constructor Method

We can create the object by using the new keyword.

Syntax : `let identifier = new Object()`

```
    identifier.key = "Value"
```

```
    identifier.key = "Value"
```

Ex:- `let details = new Object()`

```
    details.name = "Dinga"
```

```
    details.age  = 35
```

```
    details.skills = "Javascript"
```

```
    console.log(details)
```

o/p = {name: 'Dinga', age: 35, skills: 'Javascript'}

Ways to create object

3. Function Constructor Method

Function constructor is a way to create objects using a function.

To create an instance of the object, you use the new keyword.

Syntax : `function ConstructorName (parameter1, parameter2){
 this.property1=parameter1
 this.property2=parameter2
}`

`let instanceName = new ConstructorName(value1, value2);`

ConstructorName is the name of the constructor function. By convention, it starts with a capital letter.

parameter1 and **parameter2** are parameters passed when creating an instance.

Ways to create object

instanceName is the name of the new object.

value1 and **value2** are the values for parameter1 and parameter2.

Ex:

```
Ex:- function Create(fname,lname){  
    this.fname=fname  
    this.lname=lname  
}
```

```
let student1=new Create("shubham",'patil')  
let student2=new Create("Zeba","Perveen")  
console.log(student1);  
console.log(student2);
```

Ways to create object

4. Class Constructor Method

They were introduced in ES6 version of js.

Class constructor is a way to create objects using a class.

Syntax :

```
class ClassName {  
  constructor(parameter1, parameter2) {  
    this.property1 = parameter1;  
    this.property2 = parameter2;  
  }  
}  
  
const instanceName = new ClassName(value1, value2);
```

Ways to create object

Ex:

```
class Create {  
    constructor(fname, lname) {  
        this.fname = fname;  
        this.lname = lname;  
    }  
}
```

```
let student1=new Create("shubham",'patil')  
let student2=new Create("Zeba","Perveen")  
console.log(student1);  
console.log(student2);
```

Ways to create object

`ClassName` is the name of the class. By convention, it starts with a capital letter.

`parameter1` and `parameter2` are values passed when creating an instance.

`this.property1` and `this.property2` are properties of the object.

`instanceName` is the name of the new object.

`value1` and `value2` are values for `parameter1` and `parameter2`.

How to access the object

To access the object we have two ways

1. Dot Notation.
2. Box Notation.

How to access the object

1. Dot Notation.

We can access the values through keys by using the dot notation.

Syntax:- obj. key

```
Ex:- let details = {  
      name : "dinga",  
      age : 35,  
      skills : "javascript"  
    }
```

```
Console.log(details.name)
```

o/p = dinga

How to access the object

2. Box Notation.

We can access the values through keys by using the box notation.

Syntax:- `obj ['key']`

```
Ex:- let details = {  
      name: "dinga",  
      age: 35,  
      skills: "javascript"  
    }
```

```
Console.log(details['name'])
```

o/p = dinga

Note: If you want to use box notation, the key should be passed in the form of string.

Methods in object

1. `object.create()`
2. `object.assign()`
3. `objects.keys()`
4. `object.values()`
5. `object.hasOwn()`
6. `object.entries()`
7. `object.seal()`
8. `object.isSealed()`
9. `objects.freeze()`
10. `object.isFrozen()`

Methods in object

1. object.create()

The object.create() method is used to create a new object from an existing object.

Syntax:- `let identifier = object.create(object_identifier)`
`identifier.key = "Value"`

Return value: A new object with the object and properties.

Ex:-

```
let details = {  
  u_name: "dinga",  
  age: 35,  
  skills: "javascript",  
}
```

```
let person = Object.create(details)  
person.u_name = 'dingi'  
person.age = 33  
console.log(person)
```

Methods in object

2. object.assign()

The `object.assign()` method is used to copy the methods and properties of one or more objects.

Syntax:- `let identifier = object.assign(object_identifier1,object_identifier2)`

Return value: It will return the target modified object.

```
Ex:- const target = { a: 1, b: 2 };  
      const source = { b: 4, c: 5 };
```

```
      const returnedTarget = Object.assign(target,source);
```

```
      console.log(target);
```

```
output: Object { a: 1, b: 4, c: 5 }
```

Methods in object

3. object.keys()

The object.keys() method returns an array with keys of an object.

Syntax:- let identifier = object.keys(object_identifier1)

Return value: It will return the keys of an object.

```
Ex:- let details = {  
    u_name: "dinga",  
    age: 35,  
    skills: "javascript",  
}  
console.log(object.keys(details))  
output:- ['u_name', 'age', 'skills']
```

Methods in object

4. object.values()

The `object.values()` method returns an array of the property values of an object.

Syntax:- `let identifier = object.values(object_identifier1)`

Return value: It will return the value of the specified object.

```
Ex:- let details = {  
    u_name: "dinga",  
    age: 35,  
    skills: "javascript",  
}  
console.log(object.values(details))  
output:- ['dinga', 35, 'javascript']
```

Methods in object

5. object.hasOwn()

The `object.hasOwn()` method is used to check if the object has specified property or not.

Syntax:- `let identifier = object.hasOwn(object_identifier, 'prop')`

Return value: Boolean values.

```
Ex:- let details = {  
    u_name: "dinga",  
    age: 35,  
    skills: "javascript",  
}  
console.log(object.hasOwn(details, 'u_name'))  
output:- true
```

Methods in object

6. object.entries()

The `object.entries()` method returns an array of the key/value pairs of an object.

Syntax:- `let identifier = object.entries(object_identifier)`

Return value: array of key/value pairs.

```
Ex:- let details = {  
    u_name: "dinga",  
    age: 35,  
    skills: "javascript",  
}  
console.log(object.entries(details))  
output:- [['u_name', 'dings'], ['age', 35], ['skills',  
        'javascript']]
```

Methods in object

7. object.seal()

The object.seal() when we use this method we can't add the properties and we can't delete the properties but you can modify the values of an property of an object.

Syntax:- `let identifier = object.seal(object_identifier)`

Return value: it will return seal object.

Ex:-

```
let details = {  
  u_name: "dinga",  
  age: 35,  
  skills: "javascript",  
}  
console.log(object.seal(details))  
output:- {u_name: 'dinga', age: 35, skills: 'javascript'}
```

Methods in object

8. object.isSealed()

The `object.isSealed()` methods gives boolean values if an object is sealed or not.

Syntax:- `let identifier = object.isSealed(object_identifier)`

Return value: Boolean values

```
Ex:- let details = {  
    u_name: "dinga",  
    age: 35,  
    skills: "javascript",  
}  
console.log(object.isSealed(details))  
output:- true
```

Methods in object

9. object.Frozen()

The object.Frozen() when we use this method we can't add the properties and we can't delete the properties and we can't modify the values of an property of an object.

Syntax:- `let identifier = object.Frozen(object_identifier)`

Return value: it return Frozen object.

Ex:-

```
let details = {  
  u_name: "dinga",  
  age: 35,  
  skills: "javascript",  
}  
console.log(object.Frozen(details))  
output:- {u_name: 'dinga', age: 35, skills: 'javascript'}
```

Methods in object

10. object.isFrozen()

The object.isFrozen() when we use this method we can't add the properties and we can't delete the properties and we can't modify the values of an property of an object.

Syntax:- `let identifier = object.isFrozen(object_identifier)`
Return value: Boolean values.

Ex:-

```
let details = {  
  u_name: "dinga",  
  age: 35,  
  skills: "javascript",  
}  
console.log(object.isFrozen(details))  
output:- true
```

Math Object

Math:

The math object is used to perform mathematical operations like `cbrt()`, `max()`, etc...

Methods in Math Object

1.Math.max()

2.Math. min()

3.Math.ceil()

4.Math. floor()

5.Math. trunc()

6.Math. round()

7.Math. sqrt()

8.Math.cbrt()

9.Math.random()

Methods in Math Object

1.Math.max()

The Math.max() method gives the maximum value among the specified values.

Syntax:- Math.max(value1,value2,value3)

Ex:- console.log(Math.max(10,11,12))

Output:- 12

2.Math. min()

The Math. min() method gives the maximum value among the specified values.

Syntax:- Math. min(value1,value2,value3)

Ex:- console.log(Math. min(10,11,12))

Output:- 10

Methods in Math Object

3.Math.ceil()

The Math.ceil() method gives the highest value and performs the ceil operation.

Syntax:- Math.ceil(value)

Ex:- console.log(Math.ceil(10.9)) // output :- 11

console.log(Math.ceil(10.1)) // output :- 11

4.Math. floor()

The Math. floor() method gives the lowest value and performs the floor operation

Syntax:- Math. floor(value)

Ex:- console.log(Math. floor(10.9)) // output:- 10

console.log(Math. floor(10.4)) // output:- 10

Methods in Math Object

5.Math. trunc()

The Math. trunc() method removes the decimal value.

Syntax:- Math. trunc(value)

Ex:- console.log(Math. trunc(10.9)) // output :- 10

console.log(Math. trunc(10.1)) // output :- 10

6.Math. round()

The Math. round() method gives the nearest integer value.

Syntax:- Math. round(value)

Ex:- console.log(Math. round(10.9)) // output:- 11

console.log(Math. round(10.4)) // output:- 10

Methods in Math Object

7.Math. sqrt()

The Math. sqrt() method gives the square root of the specified value.

Syntax:- Math. sqrt(value)

Ex:- console.log(Math. sqrt(36)) // output :- 6

console.log(Math. sqrt(4)) // output :- 2

8.Math.cbrt()

The Math.cbrt() method gives the cube root of the specified value.

Syntax:- Math.cbrt(value)

Ex:- console.log(Math.cbrt(8)) // output:- 2

console.log(Math.cbrt(125)) // output:- 5

Methods in Math Object

9.Math.random()

The Math. random() method is used to generate the random values between 0 to 1.

Syntax:- Math.random(value)

Ex:- console.log(Math.random()) // output :- 0.1

console.log(Math.random()) // output :- 0.12345

If we want to generate random values below 100?

Ex:- let random = Math.random()*100

console.log(random) // output:- 20

console.log(random) // output:- 35

Date Object

JavaScript Date objects represent a single moment in time in a platform-independent format.

The Date object is used to work with dates and times.

It provides methods for getting and setting the year, month, day, hour, minute, second, and millisecond, as well as formatting and manipulating dates.

Ways to create Date Object

Date Constructor()

You can create date object by using new Date() Constructor

Syntax:

```
let variableName=new Date()
```

Ex:

```
let date =new Date()
```

```
console.log(date);
```

```
Mon Sep 09 2024 07:52:34 GMT+0530 (India Standard Time) // output
```

Methods of Date Object

`Date.now()` :It returns the number of milliseconds elapsed since midnight at the beginning of January 1, 1970, UTC.

Syntax:

```
let variableName=Date.now()
```

Return Type:A number representing the timestamp, in milliseconds, of the current time.

Ex:let date = Date.now()

```
console.log(date);
```

```
1725849275157 // Output (time in millisecond from 1st jan 1970 midnight upto now)
```

Methods of Date Object

`getFullYear()`: The `getFullYear()` method returns the year of a date as a four digit number:

Syntax:

```
let variableName=new Date()  
VariableName.getFullYear()
```

Return Type:An integer representing the year for the given date according to local time

Ex:

```
let date =new Date()  
console.log(date.getFullYear());
```

2024 // Output

Methods of Date Object

`getMonth()`: The `getMonth()` method returns the month of a date as a number (0-11).

Syntax:

```
let variableName=new Date()  
VariableName.getMonth()
```

Note-0 represents January.

Return Type:An integer, between 0 and 11, representing the month for the given date

Ex:

```
let date =new Date()  
  
console.log(date.getMonth());  
  
8 // Output
```

Methods of Date Object

`getDate()`: The `getDate()` method returns the day of a date as a number (1-31)

Syntax:

```
let variableName=new Date()  
VariableName.getDate()
```

Return Type: An integer, between 1 and 31, representing the day of the month

Ex:

```
let date =new Date()  
console.log(date.getDate());  
9 // Output
```

Methods of Date Object

`setFullYear()`: The `setFullYear()` method sets the year of a date object.

Syntax:

```
let variableName=new Date()  
VariableName.setFullYear()
```

Return Type:Changes the Date object in place, and returns its new timestamp.

Ex:

```
let date =new Date()  
  
date.setFullYear(2022);  
  
console.log(date);
```

Fri Sep 09 2022 08:29:53 GMT+0530 (India Standard Time) // Output

(BOM)Browser Object Model

The Browser Object model allows us to communicate with browsers.

Browser is always indicated by the **window**.

In this window, we will have many object models like document , screen , navigator , location.

Window Properties:

1. document
2. screen
3. history
4. navigator
5. location
6. innerheight
7. outerheight
8. innerwidth
9. outerwidth

Browser Object Model

1.Document:

It always represents the current html document. By using this, we can make changes in the HTML document.

2. Screen:

It provides the basic information about the screen like height , width , orientation , etc...

3. History:

It will keep a track of web pages you opened and it will store as history. We can perform forward() , back() operations using this.

4. Navigator:

It provides information about browser like appname , aversion , etc...

5. Location:

It provides the basic information about the current location like path , hostname , href , etc...

Browser Object Model

6. InnerHeight:

It calculates the browser height excluding console area.

7. OuterHeight:

It calculates the browser height including console area.

8. Innerwidth:

It calculates the browser width excluding console area.

9. Outerwidth:

It calculates the browser width including console area.

Browser Object Model

Window methods:

1. `prompt()`
2. `alert()`
3. `confirm()`
4. `open()`
5. `close()`
6. `setTimeout()`
7. `setInterval()`
8. `clearTimeout()`
9. `clearInterval()`

Browser Object Model

1. `prompt()`:

It helps to take user input. It will display a popup message in the browser.

2. `alert()`:

It will display a popup message with a button "OK".

3. `confirm()`:

It will display a popup message with "OK" & "CANCEL" buttons.

4. `open()`:

It helps to open a new webpage.

5. `close()`:

It allows us to close the current webpage.

Browser Object Model

6. `setTimeout()`:

It holds the execution until the completion of time.

Syntax: `setTimeout( callback_fn , timeout );`

7. `setInterval()`:

It will perform the same task multiple times with the time gap of interval time.

Syntax: `setInterval( callback_fn , intervaltime );`

8. `clearTimeout()`:

It will stop the working of the `setTimeout()` method.

9. `clearInterval()`:

It will stop the working of the `setInterval()` method.

Document Object Model

DOM stands for Document Object model.

DOM is a programming interface to communicate with document and javascript.

Document always represents HTML code of a web page.

By using DOM we can manipulate over the HTML tags , HTML attributes and CSS properties.

Whenever the webpage reloads it creates a document which will be stored as a tree-like structure.

Document Object Model

DOM Selectors:

These are used to target HTML elements in the document.

They are:

1. `getElementById()`
2. `getElementsByClassName()`
3. `getElementsByTagName()`
4. `getElementsByName()`
5. `querySelector()`
6. `querySelectorAll()`

Document Object Model

1. getElementById():

It will target the element based on id name.

It will return only the first matching element with a particular Id.

Syntax: `document.getElementById(" idName ");`

2. getElementsByClassName():

It will target the element based on class name.

It will return all the matching elements with a particular class name.

It will return the data in the form of HTML collection.

Syntax: `document.getElementsByClassName(" className ");`

3. getElementsByTagName():

It will target the element based on tagname.

It will return all the matching elements with a particular tag name.

It will return the data in the form of HTML collection.

Syntax: `document.getElementsByTagName(" tagName ");`

Document Object Model

4. `getElementsByName()`:

It will target the element based on the name attribute.

It will return all the matching elements with a particular name attribute.

It will return the data in the form of Node List.

Syntax: `document.getElementsByName(" name ");`

5. `querySelector()`:

It will target the elements based on id name , class name & Tag name.

It will return only the first matching element even that is id / class / tag.

Syntax: `document.querySelector(" #idName / .className / tagName ");`

6. `querySelectorAll()`:

It will target the elements based on id name , class name & Tag name.

It will return all the matching elements even that is id / class / tag.

Syntax: `document.querySelectorAll(" #idName / .className / tagName ");`

NOTE:

- Some of the above selectors will return a collection.
- These collections are not actual arrays , They are array-like collections.
- We can perform indexing & Iterations but not array methods like push. Pop , shift , unshift etc...

Document Object Model

CRUD ON HTML ELEMENTS

1.createElement():

It will create a new element.

Syntax:

```
let variableName = document.createElement("elementName");
```

Ex:

```
let div = document.createElement("div");
```

2. append():

It will append the content or node into the parent element.

Syntax:

```
parent.append(content / node)
```

Ex:

```
section.append("This is Text Information"); section.append(div);
```

3. appendChild():

It will append only nodes into the parent element.

Syntax:

```
parent.appendChild(node)
```

Ex:

```
section.appendChild("This is Text Information"); // Error  
section.appendChild(div);
```

Document Object Model

CRUD ON HTML ELEMENTS

4. `textContent()` / `innerText()` / `innerHTML()`:

All these three are used to update content in an element.
`textContent()` & `innerText()` are similar.
`innerHTML` allows us to use html tags.

Syntax:

```
selector.textContent =(content);  
selector.innerText =(content);  
selector.innerHTML =(content);
```

Ex:

```
div.textContent = "This is content";  
div.innerText = "This is content";  
div.innerHTML = <b>"This is content"</b>;
```

5. `replaceChild()`:

It allows us to replace the node in a document with a new node.

Syntax:

```
parent.replaceChild(newNode , prevNode)
```

Ex:

```
section.replaceChild(h1, div);
```

Document Object Model

CRUD ON HTML ELEMENTS

6. removeChild():

It will remove the node from the element.

Syntax: parent.removeChild(nodeName)

Ex: section.removeChild(h1);

Document Object Model

CRUD ON HTML ATTRIBUTES

1. `setAttribute()`:

It will add a new attribute to the element.

If the attribute is already present it will update the attribute value.

Syntax:

```
selector.setAttribute("attributeName","attributeValue")
```

Ex:

```
div.setAttribute("id","demo");
```

2. `getAttribute()`:

It will return the attribute value of an element.

Syntax:

```
selector.getAttribute("attributeName")
```

Ex:

```
div.getAttribute("id"); // demo
```

3. `removeAttribute()`:

It will remove the existing attribute of an element.

Syntax:

```
selector.removeAttribute("attributeName")
```

Ex:

```
div.removeAttribute("id");
```

Document Object Model

CRUD ON CSS PROPERTIES

1. `setProperty()` , `propertyName` , `cssText`:

All these are used to add CSS properties to an element.

By using `setProperty` & `propertyName`, we can add only property at a time.

By using `cssText`, we can add multiple properties at a time.

it works as inline style only

Syntax:

```
selector.style.setProperty("property-name","propertyName")
selector.style.propertyName = "propertyValue";
selector.style.cssText = "property1: value1; property2: value2; ..."; //?
```

Ex:

```
div.style.setProperty("font-size","40px");
div.style.fontSize = "40px";
div.style.cssText = "font-size: 40px";
```

Document Object Model

CRUD ON CSS PROPERTIES

2. `getPropertyValue()`:

It allows us to get the value of a property of an element.

Syntax:

```
selector.style.getPropertyValue("propertyName")
```

Ex:

```
div.style.getPropertyValue("font-size");
```

3. `removeProperty()`:

It allows us to remove the css property of an element.

Syntax:

```
selector.style.removeProperty("propertyName")
```

Ex:

```
div.style.removeProperty("font-size");
```

Event

Events are the actions that are performed by either browser or user.
We have many events, some of the events are

Mouse Events	Key Events	Form Events	Window Events
click dblclick mouseup mousedown mouseover mouseout mousemove	keypress keydown keyup	submit reset focus blur change	load resize scroll

Event

We can attach these events to the html element in 3 ways. They are:

1. HTML Attributes
2. DOM Properties
3. addEventListeners

HTML Attributes:

In this case we have to use events as attributes.

Syntax:

```
<tagname onEventName=" "> </tagname>
```

Ex :

```
<div onclick="alert('Click Event is triggered')"> Div Tag </div>
```

Event

DOM Properties:

In this case, we use DOM Properties of an element to attach events to an element.

Syntax:

```
Element.onEventName = function
```

Ex:

```
div.onclick = fun;  
function fun() {  
    div.style.cssText = "color: white ; background-color: teal; border:1px  
    solid;text-align:center;  
}
```

addEventListener :

This is the best way to attach events to the elements.

By using html attributes & Dom Properties, we can use one event only once.

But by using addEventListener we can attach multiple functions to the same event.

Syntax:

```
Element.addEventListener("eventName" , function )
```

Event

Ex:

```
button.addEventListener("click", () => {  
    console.log("Click Event is Triggered")  
});
```

If we need to remove the event from an element we can use `removeEventListener`.

Ex:

```
button.addEventListener("click", greet);  
  
function greet() {  
    console.log("Hi Hello How are you?");  
}  
  
button.removeEventListener("click", greet);
```